

Assignment 3 Report

CS334: Operating Systems Lab

Scheduling Algorithms

Team Members:

Maddala Bogadi Sushanth (Roll No. 230101061)
Varshith Grandhi (Roll No. 230101041)
Akshay Bobbala (Roll No. 230101009)
Muduguru Mahedhar Reddy (Roll No. 230101067)

Contents

1	Task 3.1: Implement kernel primitives for IPC(Inter Process Communication)	3
1.1	Introduction	3
1.2	Implementation Methodology	3
1.2.1	System Calls	4
1.2.2	Shared Memory	4
1.2.3	Mailboxes	4
1.3	Testing and Results	5
1.4	Conclusion	5
2	Task 3.2: The Intertwined Memory Challenge	8
2.1	Introduction	8
2.2	Implementation	8
2.2.1	Master Program	8
2.2.2	Path Structure	9
2.3	Explanation of Communication Protocol	9
2.4	Deadlock Avoidance Strategy	10
2.5	Results	10
2.6	Conclusion	10

1 Task 3.1: Implement kernel primitives for IPC(Inter Process Communication)

1.1 Introduction

Default Xv6 kernel's IPC capabilities are limited. The objective of this task was to implement two core primitives to enable more complex process coordination:

1. shared memory for efficient data sharing
2. mailboxes for reliable messaging between processes.

1.2 Implementation Methodology

We modified xv6 by adding system calls for shared memory using `kalloc()` function to allocate pages, `mappages()` to map page to user-space and we created a global table to keep track of active shared memory regions, reference counter for each region.

We implemented mailboxes as a global, fixed-size circular queue, using the `sleep` and `wakeup` mechanisms to handle blocking for send and receive operations. Deadlock conditions were also properly managed.

Global Array Declaration for shared memory

We created `struct mem_reg` in `shm.h`, which contains a pointer to the physical address, a count of the number of processes holding the shared memory, a spinlock.

Global Array Declaration for Mailboxes

We created `struct mbox` in `mbox.h`, which contains a circular queue, a spinlock and a valid bit.

1.2.1 System Calls

Six system calls were introduced:

- `shm_create(int key)`: Creates a new shared memory region of one page associated with an integer key. If a region with this key already exists, it should not create a new one. It returns a handle or identifier for this region.
- `shm_get(int key)`: Allows a process to access (or "attach") to the shared memory region identified by key. This call should map the physical page associated with the key into the process's virtual address space and return a pointer to it. All processes that call `shm_get` with the same key should receive a pointer to the same physical page.
- `shm_close(int key)`: Detaches the shared memory region from the current process's address space. The physical page should only be freed when the last process attached to it calls `close`.
- `mbox_create(int key)`: Creates a message queue (mailbox) identified by a key. This mailbox will be used for passing integer messages
- `mbox_send(int mbox_id, int msg)`: Sends an integer message `msg` to the mailbox identified by `mbox_id`. If the mailbox is full, the process should block until space becomes available.
- `mbox_receive(int mbox_id, int *msg)`: Receives an integer message from the mailbox `mbox_id` and stores it in the location pointed to by `msg`. If the mailbox is empty, the process should block until a message arrives.

The necessary changes were made in `shm.c`, `shm.h`, `mbox.c`, `mbox.h`, `defs.h`, `user.h`, and `usys.pl`. The implementations were added in `sysproc.c`, with proper locking to ensure consistency.

1.2.2 Shared Memory

For two processes try to use common variable, we use shared memory. It allows two or more processes access the same region of memory during IPC. `shm_create(int key)` allocates the memory(page) for a shared region, processes can access it by `shm_get(int key)` and get virtual address of that page. `shm_close(int key)` removes the access of shared memory for that process, and in case no process use the shared region, it deallocates the memory using `uvmunmap()` function.

1.2.3 Mailboxes

When two cooperative processes need to communicate for control of variables, mailbox acts as a buffer where messages are stored until receiver picks them up. `mbox_create(int key)` allocates a circular queue(struct `mbox`) to the process

`mbox_send(int mbox_id, int msg)`: Acquires the lock – ensures only one process can modify the mailbox queue at a time.

Try to enqueue – if the mailbox is full (enqueue returns -1)

Wake up anyone sleeping on this mailbox (likely a receiver waiting for messages).
 Sleep on the mailbox. Crucially, sleep releases the lock while sleeping, so other processes can access the mailbox.
 Once space is available, the message is enqueued.
 Wake up other processes that might be sleeping (like other senders or receivers).
 Release the lock.

`mbox_receive(int mbox_id, int *msg)`: Acquire the lock – only one receiver can access the mailbox queue at a time.

Try to dequeue – if the mailbox is empty (dequeue returns -1):
 Wake up anyone sleeping (likely senders waiting to put messages).
 Sleep on the mailbox, releasing the lock while sleeping.
 Once a message is available, it is dequeued and returned.
 Wake up other processes that might be waiting.
 Release the lock.

Handling Deadlock

- Both senders and receivers acquire the lock to access the mailbox (Mutual Exclusion).
- If a sender or receiver cannot proceed because the mailbox is full or empty, it sleeps and releases the lock, allowing other processes to make progress.
- Whenever a process sleeps, it first calls `wakeup` to notify any waiting process.
- A sleeping sender does not hold the lock, and a sleeping receiver does not hold the lock, preventing circular wait.
- If the mailbox is full, at least one receiver can dequeue a message, freeing space for senders.
- If the mailbox is empty, at least one sender can enqueue a message, producing a message and ensuring progress.

1.3 Testing and Results

A test program `mboxtest.c` and `shmttest.c` was written to validate the implementation.

The output clearly demonstrates correct functioning of system calls: the child modifies the shared variable with its parent in the shared memory output. In the mailbox test, the child process sends 5 messages to the mailbox and the parent receives the messages from their shared mailbox.

1.4 Conclusion

We implemented Inter Process Communication in xv6 by two methods, shared memory and mailboxes. Default xv6 has limited IPC, which is now enhanced. Now two or more processes can communicate to each other by common variables or by sending/receiving

```
hart 2 starting
hart 1 starting
init: starting sh
$ shmtest
shmtest: starting...
shmtest: writing to shared memory
child: read from shared memory: hello from proc1
parent: read from shared memory: hello from child
shmtest: finished.
$
```

Figure 1.1: Output of `shmtest.c`

```
init: starting sh
$ mboxtest
child:sending0
child:sending1
child:sending2
child:sending3
child:sending4
parent:received0
parent:received1
parent:received2
parent:received3
parent:received4
$
```

Figure 1.2: Output of `mboxtest.c`

messages from mailboxes which enables communication and synchronization. Synchronization makes the processes to run in order.

```

1  name & mbox; @ mbox mbox;
2  #include "types.h"
3  #include "kernel.h"
4  #include "mmaplayout.h"
5  #include "fs.h"
6  #include "spinlock.h"
7  #include "types.h"
8  #include "mfs.h"
9  #include "mbox.h"
10 #define NULL (void*)0
11
12
13 #define size 16
14 #define mbox_size 2000
15 #define mbox_array of mbox_size mbox;
16 int enqueue(int val, struct mbox *buffer)
17 {
18     if (buffer->length==size)
19         return 0;
20     if (buffer->front==buffer->front+1)
21         buffer->rear=(buffer->rear+1)%size;
22     buffer->enqueue(buffer->rear+val);
23     buffer->length++;
24     return 0;
25 }
26
27 int dequeue(struct mbox *buffer)
28 {
29     int val;
30     if (buffer->length==0) return -1;
31     if (buffer->front==buffer->rear+1)
32         val=buffer->dequeue(buffer->front);
33         buffer->rear--;
34         buffer->front--;
35     }
36     else
37         val=buffer->dequeue(buffer->front);
38         buffer->front=buffer->front+1;
39     }
40     buffer->length--;
41     return val;
42 }
43
44 void mbox_init()
45 {
46     for (int i=0; i<mbox_size; i++)
47         array of mbox[i].front=-1;
48     array of mbox[i].rear=-1;
49     array of mbox[i].length=0;
50     array of mbox[i].val=-1;
51     }
52     initlock(array of mbox[i].lock, "buffer lock");
53 }
54
55 int mbox_create(int key)
56 {
57     if (array of mbox[key].val!=0) return -1;
58     else array of mbox[key].val=1;
59     return 0;
60 }
61
62
63
64 int mbox_send(int mbox_id, int msg)
65 {
66     acquire(array of mbox_id).lock);
67     while (enqueue(array of mbox_id).mbox_id) == -1)
68         sleep(array of mbox_id).lock);
69     sleep(array of mbox_id).lock);
70     enqueue(array of mbox_id).msg);
71     release(array of mbox_id).lock);
72     return 0;
73 }
74
75 int mbox_recv(int mbox_id, int *msg)
76 {
77     acquire(array of mbox_id).lock);
78     int temp;
79     while (dequeue(array of mbox_id).mbox_id) == -1)
80         sleep(array of mbox_id).lock);
81     sleep(array of mbox_id).lock);
82     temp=dequeue(array of mbox_id).msg);
83     release(array of mbox_id).lock);
84     return 0;
85 }

```

Figure 1.3: mbox.c

```

1  #include <stdio.h>
2  #include <sys/socket.h>
3  #include "types.h"
4  #include "param.h"
5  #include "memlayout.h"
6  #include "riscv.h"
7  #include "spinlock.h"
8  #include "sleep.h"
9  #include "defs.h"
10 #include "user.h"
11
12 #define PAGE_SIZE 1024
13
14 struct mem_req {
15     struct spinlock lock;
16     int key;
17     int value;
18 };
19
20 void mem_init() {
21     for (int i = 0; i < NMEM; i++) {
22         struct mem_req *m = &mem[i];
23         spinlock_init(&m->lock);
24         m->key = i;
25         m->value = 0;
26     }
27 }
28
29 int mem_create(int key) {
30     struct mem_req *m;
31     spinlock_acquire(&m->lock);
32     if (m->key == -1) {
33         m->key = key;
34         m->value = 0;
35     } else {
36         return -1;
37     }
38     return 0;
39 }
40
41 void mem_get(int key) {
42     struct mem_req *m;
43     spinlock_acquire(&m->lock);
44     if (m->key == -1) {
45         return -1;
46     }
47     printf("mem_get: key=%d, value=%d\n", key, m->value);
48 }
49
50 int mem_set(int key, int value) {
51     struct mem_req *m;
52     spinlock_acquire(&m->lock);
53     if (m->key == -1) {
54         return -1;
55     }
56     m->value = value;
57     return 0;
58 }
59
60 int mem_close(int key) {
61     struct mem_req *m;
62     spinlock_acquire(&m->lock);
63     if (m->key == -1) {
64         return -1;
65     }
66     m->key = -1;
67     return 0;
68 }
69
70 int mem_test(int key) {
71     struct mem_req *m;
72     spinlock_acquire(&m->lock);
73     if (m->key == -1) {
74         return -1;
75     }
76     return m->value;
77 }
78
79 int mem_test_set(int key, int value) {
80     struct mem_req *m;
81     spinlock_acquire(&m->lock);
82     if (m->key == -1) {
83         return -1;
84     }
85     m->value = value;
86     return 0;
87 }
88
89 int mem_test_close(int key) {
90     struct mem_req *m;
91     spinlock_acquire(&m->lock);
92     if (m->key == -1) {
93         return -1;
94     }
95     m->key = -1;
96     return 0;
97 }
98
99 int mem_test_get(int key) {
100    struct mem_req *m;
101    spinlock_acquire(&m->lock);
102    if (m->key == -1) {
103        return -1;
104    }
105    return m->value;
106 }
107
108 int mem_test_set_get(int key, int value) {
109    struct mem_req *m;
110    spinlock_acquire(&m->lock);
111    if (m->key == -1) {
112        return -1;
113    }
114    m->value = value;
115    return m->value;
116 }
117
118 int mem_test_close_get(int key) {
119    struct mem_req *m;
120    spinlock_acquire(&m->lock);
121    if (m->key == -1) {
122        return -1;
123    }
124    m->key = -1;
125    return 0;
126 }
127
128 int mem_test_get_close(int key) {
129    struct mem_req *m;
130    spinlock_acquire(&m->lock);
131    if (m->key == -1) {
132        return -1;
133    }
134    m->key = -1;
135    return m->value;
136 }
137
138 int mem_test_set_close(int key, int value) {
139    struct mem_req *m;
140    spinlock_acquire(&m->lock);
141    if (m->key == -1) {
142        return -1;
143    }
144    m->key = -1;
145    m->value = value;
146    return 0;
147 }
148
149 int mem_test_get_set(int key, int value) {
150    struct mem_req *m;
151    spinlock_acquire(&m->lock);
152    if (m->key == -1) {
153        return -1;
154    }
155    m->key = -1;
156    m->value = value;
157    return m->value;
158 }
159
160 int mem_test_set_set(int key, int value) {
161    struct mem_req *m;
162    spinlock_acquire(&m->lock);
163    if (m->key == -1) {
164        return -1;
165    }
166    m->key = -1;
167    m->value = value;
168    return m->value;
169 }
170
171 int mem_test_close_set(int key, int value) {
172    struct mem_req *m;
173    spinlock_acquire(&m->lock);
174    if (m->key == -1) {
175        return -1;
176    }
177    m->key = -1;
178    m->value = value;
179    return 0;
180 }
181
182 int mem_test_get_set_close(int key, int value) {
183    struct mem_req *m;
184    spinlock_acquire(&m->lock);
185    if (m->key == -1) {
186        return -1;
187    }
188    m->key = -1;
189    m->value = value;
190    return m->value;
191 }
192
193 int mem_test_set_set_close(int key, int value) {
194    struct mem_req *m;
195    spinlock_acquire(&m->lock);
196    if (m->key == -1) {
197        return -1;
198    }
199    m->key = -1;
200    m->value = value;
201    return m->value;
202 }
203
204 int mem_test_get_set_set(int key, int value) {
205    struct mem_req *m;
206    spinlock_acquire(&m->lock);
207    if (m->key == -1) {
208        return -1;
209    }
210    m->key = -1;
211    m->value = value;
212    return m->value;
213 }
214
215 int mem_test_set_set_set(int key, int value) {
216    struct mem_req *m;
217    spinlock_acquire(&m->lock);
218    if (m->key == -1) {
219        return -1;
220    }
221    m->key = -1;
222    m->value = value;
223    return m->value;
224 }
225
226 int mem_test_get_set_set_close(int key, int value) {
227    struct mem_req *m;
228    spinlock_acquire(&m->lock);
229    if (m->key == -1) {
230        return -1;
231    }
232    m->key = -1;
233    m->value = value;
234    return m->value;
235 }
236
237 int mem_test_set_set_set_close(int key, int value) {
238    struct mem_req *m;
239    spinlock_acquire(&m->lock);
240    if (m->key == -1) {
241        return -1;
242    }
243    m->key = -1;
244    m->value = value;
245    return m->value;
246 }
247
248 int mem_test_get_set_set_set(int key, int value) {
249    struct mem_req *m;
250    spinlock_acquire(&m->lock);
251    if (m->key == -1) {
252        return -1;
253    }
254    m->key = -1;
255    m->value = value;
256    return m->value;
257 }
258
259 int mem_test_set_set_set_set(int key, int value) {
260    struct mem_req *m;
261    spinlock_acquire(&m->lock);
262    if (m->key == -1) {
263        return -1;
264    }
265    m->key = -1;
266    m->value = value;
267    return m->value;
268 }
269
270 int mem_test_get_set_set_set_close(int key, int value) {
271    struct mem_req *m;
272    spinlock_acquire(&m->lock);
273    if (m->key == -1) {
274        return -1;
275    }
276    m->key = -1;
277    m->value = value;
278    return m->value;
279 }
280
281 int mem_test_set_set_set_set_close(int key, int value) {
282    struct mem_req *m;
283    spinlock_acquire(&m->lock);
284    if (m->key == -1) {
285        return -1;
286    }
287    m->key = -1;
288    m->value = value;
289    return m->value;
290 }
291
292 int mem_test_get_set_set_set_set(int key, int value) {
293    struct mem_req *m;
294    spinlock_acquire(&m->lock);
295    if (m->key == -1) {
296        return -1;
297    }
298    m->key = -1;
299    m->value = value;
300    return m->value;
301 }
302
303 int mem_test_set_set_set_set_set(int key, int value) {
304    struct mem_req *m;
305    spinlock_acquire(&m->lock);
306    if (m->key == -1) {
307        return -1;
308    }
309    m->key = -1;
310    m->value = value;
311    return m->value;
312 }
313
314 int mem_test_get_set_set_set_set_close(int key, int value) {
315    struct mem_req *m;
316    spinlock_acquire(&m->lock);
317    if (m->key == -1) {
318        return -1;
319    }
320    m->key = -1;
321    m->value = value;
322    return m->value;
323 }
324
325 int mem_test_set_set_set_set_set_close(int key, int value) {
326    struct mem_req *m;
327    spinlock_acquire(&m->lock);
328    if (m->key == -1) {
329        return -1;
330    }
331    m->key = -1;
332    m->value = value;
333    return m->value;
334 }
335
336 int mem_test_get_set_set_set_set_set(int key, int value) {
337    struct mem_req *m;
338    spinlock_acquire(&m->lock);
339    if (m->key == -1) {
340        return -1;
341    }
342    m->key = -1;
343    m->value = value;
344    return m->value;
345 }
346
347 int mem_test_set_set_set_set_set_set(int key, int value) {
348    struct mem_req *m;
349    spinlock_acquire(&m->lock);
350    if (m->key == -1) {
351        return -1;
352    }
353    m->key = -1;
354    m->value = value;
355    return m->value;
356 }
357
358 int mem_test_get_set_set_set_set_set_close(int key, int value) {
359    struct mem_req *m;
360    spinlock_acquire(&m->lock);
361    if (m->key == -1) {
362        return -1;
363    }
364    m->key = -1;
365    m->value = value;
366    return m->value;
367 }
368
369 int mem_test_set_set_set_set_set_set_close(int key, int value) {
370    struct mem_req *m;
371    spinlock_acquire(&m->lock);
372    if (m->key == -1) {
373        return -1;
374    }
375    m->key = -1;
376    m->value = value;
377    return m->value;
378 }
379
380 int mem_test_get_set_set_set_set_set_set(int
```

Figure 1.4: shm.c

```
kernel >  mbox.h >  lock
1  #include "types.h"
2  #include "spinlock.h"
3
4  struct mbox
5  {
6      int queue[10];
7      int front;
8      int rear;
9      int length;
10     int valid;
11     struct spinlock lock;
12 };
13
14 #define max_mboxes 1000
15 extern struct mbox array_of_mb[max_mboxes];
16 int enqueue(int, struct mbox*);
17 int dequeue(struct mbox*);
18 void mbox_init(void);
19 int mbox_create(int);
20 int mbox_send(int, int);
21 int mbox_recv(int, int*);
```

Figure 1.5: mbox.h

```
kernel > c shm.h > ③ shm_create(int)
1  #include "types.h"
2  #include "spinlock.h"
3
4  #define NULL ((void*)0)
5
6  struct mem_reg{
7      void *pa;
8      int no_of_process;
9      struct spinlock lock;
10 };
11
12 #define physical_pages 1000
13 extern struct mem_reg gt[physical_pages];
14
15 // shm.c
16 void shm_init(void);
17 int shm_create(int);
18 void* shm_get(int);
19 int shm_close(int);
```

Figure 1.6: shm.h

2 Task 3.2: The Intertwined Memory Challenge

2.1 Introduction

In this task, we use the primitives created in Task 1 to build a concurrent application where a team of two processes must communicate and cooperate to navigate a shared data structure. Applying our understanding of shared memory and message-passing to solve a synchronization and data-sharing problem. The challenge involves two processes that navigate an "intertwined" path stored in shared memory, where neither process knows its own path and must rely on other process for directions.

2.2 Implementation

2.2.1 Master Program

This program sets up the game.

1. `shm_create()` to allocate the shared memory page. It initializes the intertwined paths within the shared memory
2. `mbox_create()` to create two mailboxes for each team: one for P1 to send to P2, and another for P2 to send to P1.
3. `fork()` to create the two processes, in each child process we call `process.c` using `exec syscall`.
4. we call `shm_close()` to deallocate the shared memory.

Process Program

This program contains the logic for a single process.

1. Attach to the shared memory using `shm_get()`.
2. Using `atoi()` to convert the string argument to integer.
3. Navigate the path step-by-step using a strict communication protocol.
4. Send its current location to another process using `mbox_send()`.
5. Wait for and receive another process's current location using `mbox_recv()`.

6. Use the location received from another process to look up its own next location in the shared memory.
7. Update its local position and repeat the loop.
8. We call `shm_close()` to deallocate the shared memory.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int main() {
    int *ptr;
    ptr = (int*)shm_get(1);
    ptr[0] = 5;
    ptr[1] = 6;
    ptr[6] = 10;
    ptr[5] = 11;
    ptr[11] = 20;
    ptr[10] = 21;
    mbox_create(1);
    mbox_create(2);
    int pid = fork();
    if (pid == 0) {
        char * args[] = {"process", "0", "21", 0};
        exec("process", args);
        exit(0);
    }
    int pid2 = fork();
    if (pid2 == 0) {
        char * args[] = {"process", "1", "20", 0};
        exec("process", args);
        exit(0);
    }
    wait(0);
    shm_close(1);
    return 0;
}
```

Figure 2.1: master.c

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int main(int argc, char *argv[]) {
    int* ptr = (int*)shm_get(1);
    int start_point = atoi(argv[1]);
    int end_point = atoi(argv[2]);
    int next_pos;
    if (start_point == 0) {
        printf("process A currently at: %d\n", start_point);
        mbox_send(1, ptr[start_point]);
        mbox_recv(2, &next_pos);
        while (next_pos != end_point) {
            printf("process A currently at: %d\n", next_pos);
            mbox_send(1, ptr[next_pos]);
            mbox_recv(2, &next_pos);
        }
    }
    else if (start_point == 1) {
        printf("process B currently at: %d\n", start_point);
        mbox_send(2, ptr[start_point]);
        mbox_recv(1, &next_pos);
        while (next_pos != end_point) {
            printf("process B currently at: %d\n", next_pos);
            mbox_send(2, ptr[next_pos]);
            mbox_recv(1, &next_pos);
        }
    }
    shm_close(1);
    return 0;
}
```

Figure 2.2: process.c

2.2.2 Path Structure

```
Process A path (hidden from A):
B positions: 1 → 5 → 10 → 20
Next A positions (revealed via B):
0 → 6 → 11 → 21

Process B path (hidden from B):
A positions: 0 → 6 → 11 → 21
Next B positions (revealed via A):
1 → 5 → 10 → 20
```

Figure 2.3: Path structure

2.3 Explanation of Communication Protocol

The intertwined path is stored in shared memory. Both the processes use mailboxes for the inter process communication.

Process A sends the next position of Process B through mailbox with key 1 and receives its next position from mailbox with key 2.

2.4 Deadlock Avoidance Strategy

- A deadlock will occur if both the processes wait to receive a message before sending one. Therefore, in our process implementation, both the processes send messages first before waiting to receive. So before every receive there is a send. This strategy avoids the deadlock.
- Alternate strategy : establish a fixed order, 1 designated process always sends first and then receives later. The other process receives first and sends later. This strategy also avoids deadlock.
- The only case a deadlock occurs is if both process try to receive first before sending.

2.5 Results

```
xv6 kernel is booting
init: starting sh
$ master
process A currently at:0
sent 5 in mailbox with id: 1
process B currently at:1
sent 6 in mailbox with id: 2
receieved 5 in mailbox with id: 1
process B currently at:5
sent 11 in mailbox with id: 2
receieved 6 in mailbox with id: 2
process A currently at:6
sent 10 in mailbox with id: 1
receieved 11 in mailbox with id: 2
process A currently at:11
sent 20 in mailbox with id: 1
receieved 10 in mailbox with id: 1
process B currently at:10
sent 21 in mailbox with id: 2
receieved 20 in mailbox with id: 1
receieved 21 in mailbox with id: 2
$ █
```

Figure 2.4: Traversal in action

2.6 Conclusion

We implemented a message sharing and message passing to solve a synchronization problem and data-sharing problem.